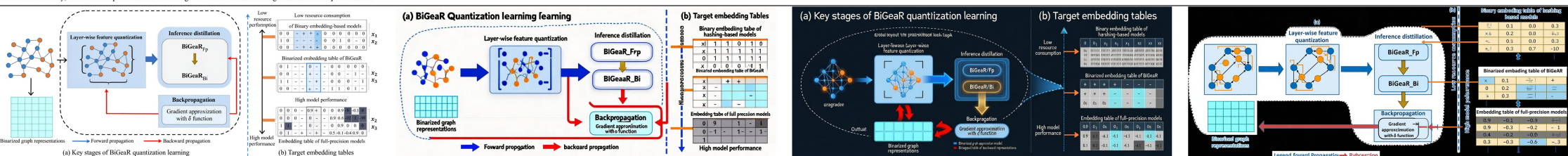
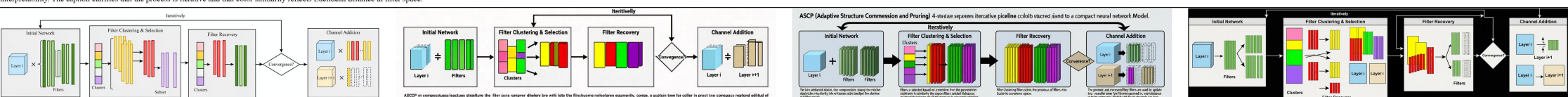


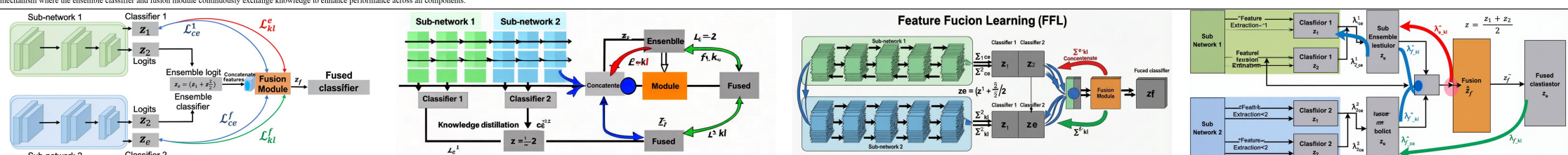
The figure illustrates the BiGeAr quantization learning framework, divided into two main parts: (a) Key stages of BiGeAr quantization learning, and (b) Target embedding tables. In part (a), the global layout is a left-to-right forward propagation flow followed by a backward propagation loop, enclosed within a dashed oval. The input is a graph structure represented by blue and orange nodes connected by edges. This input flows into a module labeled 'Layer-wise feature quantization', depicted as a light blue rectangular box containing a smaller graph with similar node colors and surrounding brackets indicating quantized features at each layer. From this module, a thick blue arrow points to the next stage, 'Inference distillation', which contains two nested rounded rectangles: the top one labeled 'BiGeAr_Fp' (full-precision model) and the bottom one 'BiGeAr_Bi' (binarized model), connected by a downward gray arrow indicating knowledge transfer. Below these, a separate light blue box labeled 'Backpropagation' contains a rounded rectangle with the text 'Gradient approximation with function'. Red arrows point from this box toward the next stage, 'Inference distillation', forming a feedback loop. A legend at the bottom clarifies that blue arrows represent forward propagation and red arrows represent backward propagation. The output, shown on the far left below the input, is a grid of light cyan squares labeled 'Binarized graph representations'. In part (b), a vertical axis labeled 'Low resource consumption' at the top and 'High model performance' at the bottom is shown. Three embedded tables are aligned along this axis, each with an orange horizontal line connecting it to the axis. The top table, titled 'Binary embedding table of hashing-based models', contains binary values (0, 1). The middle table, titled 'Binarized embedding table of BiGeAr', contains values of '+' and '-' for the same inputs, with some cells highlighted in light blue. The bottom table, titled 'Embedding table of full-precision models', contains floating-point numbers (e.g., 0.9, -0.1, etc.) for the same inputs, with some cells highlighted in dark gray. The overall structure emphasizes the trade-off between computational efficiency and model accuracy, with BiGeAr positioned as achieving a favorable balance through its binarized representation.



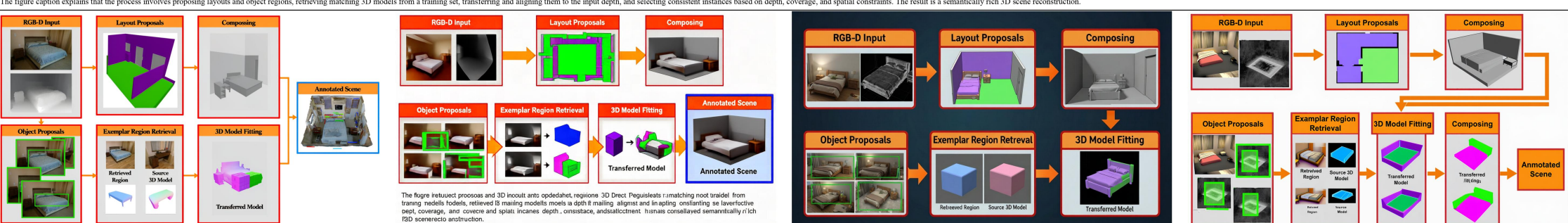
The figure presents an overview of the ASCP (Adaptive Structure Compression and Pruning) method, structured as a four-stage iterative pipeline leading to a compact neural network model. The global layout is left-to-right, with a feedback loop from the final stage back to the second stage, controlled by a convergence check. The entire process is labeled 'Iteratively' at the top, indicating repeated execution until convergence. The first module, labeled 'Initial Network', contains a light blue cube representing 'Layer 1' connected via a multiplication symbol to multiple vertical rectangular bars grouped into sets, representing 'Filters'. These filters are shown in varying shades of green, indicating different filter types or initial states. This module serves as the starting point for the compression process. The second module, 'Filter Clustering & Selection', receives input from the initial network. It features a vertical stack of colored squares on the left, labeled 'Clusters', with colors including pink, yellow, green, and purple. These correspond to clusters of filters, which are visually represented as groups of vertical bars in matching colors (e.g., yellow bars grouped together, red bars together, etc.). Arrows point from the cluster colors to their respective filter groups, indicating assignment. The module shows that filters are grouped based on similarity, with similar colors implying proximity in Euclidean space, as noted in the caption. A subset of these clusters is selected for retention. The third module, 'Filter Recovery', takes the selected filters from the previous step. It displays the same colored filter groups (yellow, red, green, purple), now arranged in a more compact form, suggesting that the selected filters have been retained or refined. This step is crucial for maintaining performance after pruning. An arrow leads from 'Filter Recovery' to a diamond-shaped decision node labeled 'Convergence?'. If convergence is not achieved, a feedback loop returns to the 'Filter Clustering & Selection' module, allowing the process to repeat. If convergence is reached, the process proceeds to the final module. The fourth module, 'Channel Addition', shows two cubes: a light blue 'Layer 1' and a beige 'Layer 1+1', each multiplied by a set of filtered channels. The channels are represented as dashed outlines of the original filter groups, now reduced in number and aligned with the corresponding layer. This indicates that the pruned and recovered filters are used to update the network structure, resulting in a more compact model with fewer channels per layer. All modules are enclosed in light gray boxes with black borders, and the connections between them are solid black arrows, except for the feedback loop, which is also a solid black arrow curving upward. The visual design uses color coding consistently to represent filter similarity and cluster membership, enhancing interpretability. The caption clarifies that the process is iterative and that color similarity reflects Euclidean distance in filter space.



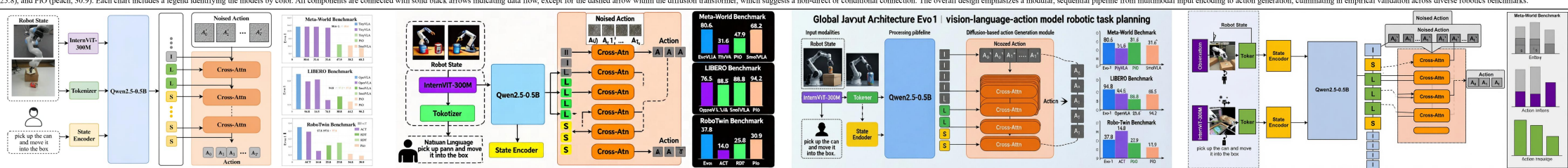
The figure illustrates the Feature Fusion Learning (FFL) framework, which consists of two parallel sub-networks—Sub-network 1 (depicted in light green) and Sub-network 2 (depicted in light blue)—each composed of multiple stacked convolutional or feature extraction blocks connected sequentially by black arrows. Each sub-network feeds into its own classifier: Classifier 1 (associated with Sub-network 1) and Classifier 2 (associated with Sub-network 2), both represented as gray rectangular blocks labeled z_1 and z_2 respectively. These classifiers output logits, which are combined via an ensemble logit computation denoted as $z = (z_1 + z_2)/2$, forming the Ensemble classifier (also a gray block labeled z). From this ensemble classifier, features are routed through a Concatenate operation (shown as a dual-colored block with a blue circle indicating merging) into a Fusion Module (an orange rectangular block). The Fusion Module processes the concatenated features and outputs z_f , which is fed into the Fused classifier (another gray block). The entire architecture employs multiple loss functions: cross-entropy losses $\mathcal{L}_{ce}$ and $\mathcal{L}_{kl}$ are applied to Classifier 1 and Classifier 2 respectively; $\mathcal{L}_{ce}$ is applied to the Fused classifier; and knowledge distillation losses $\mathcal{L}_{kd}$ and $\mathcal{L}_{kl}$ are used for bidirectional knowledge transfer. Specifically, $\mathcal{L}_{ce}$ (red curved arrow) represents knowledge distillation from the Ensemble classifier to the Fusion Module, while $\mathcal{L}_{kl}$ (green curved arrow) denotes reverse knowledge distillation from the Fused classifier back to both Classifier 1 and Classifier 2. Blue curved arrows indicate forward propagation paths from individual classifiers to the Concatenate block, while black arrows denote standard feed-forward connections within each sub-network and between classifiers and the ensemble. The diagram emphasizes an online mutual knowledge distillation mechanism where the ensemble classifier and fusion module continuously exchange knowledge to enhance performance across all components.



The figure illustrates a complete pipeline for 3D scene understanding and annotation from RGB-D input, structured as a flowchart with six main processing stages arranged in two parallel paths that converge at the final output. The global layout is horizontal and hierarchical: the top row shows the layout estimation and composition path, while the bottom row details the object detection, retrieval, and model fitting path. Both paths originate from the same 'RGB-D Input' block and feed into the final 'Composing' stage, which produces the 'Annotated Scene'. The top path starts with 'Layout Proposals' (showing a 3D scene with colored bounding boxes) and 'Composing' (showing the final annotated scene). The bottom path starts with 'Object Proposals' (showing 3D models of objects) and '3D Model Fitting' (showing the objects being fitted into the scene). The middle path involves 'Exemplar Region Retrieval' (showing retrieved regions from a database) and '3D Model Fitting' (showing the objects being fitted into the scene). The diagram uses color coding to represent different stages: red for input/output, green for layout estimation, blue for object detection, and orange for model fitting. The caption explains that the process involves proposing layouts and object regions, retrieving matching 3D models from a training set, transferring and aligning them to the input depth, and selecting consistent instances based on depth, coverage, and spatial constraints. The result is a semantically rich 3D scene reconstruction.



The figure illustrates the architecture of Evo-1, a vision-language-action model for robotic task planning. The global layout is horizontally structured into four main sections: input modalities on the far left, a central processing pipeline, a diffusion-based action generation module, and benchmark performance results on the far right. On the left, two types of inputs are shown: visual observations and language instructions. The visual input consists of two RGB images depicting a robotic arm interacting with objects (e.g., a can and a box), enclosed within a dashed rectangle labeled 'Robot State'. Below this, a human icon provides a natural language instruction: 'pick up the can and move it into the box', also enclosed in a dashed rectangle. These inputs feed into the central processing pipeline. The visual input passes through a 'Tokenizer' and a 'State Encoder' to produce tokens. The language input passes through a 'Tokenizer' and a 'State Encoder' to produce tokens. The outputs from these encoders converge into a large light-blue rectangular block labeled 'Qwen2.5-0.5B', representing the core vision-language model. From this block, the pipeline branches into two paths: one for action generation and one for policy learning. The action generation path involves a 'Noised Action' module, which generates a sequence of actions (e.g., $A_1, A_2, \dots, A_n$). The policy learning path involves a 'Policy Network' that takes the robot state and the generated actions to produce a policy. The benchmark performance results on the far right show the model's performance on various tasks, including 'Meta-World Benchmark', 'LIBERO Benchmark', and 'RoboTwin Benchmark'. The diagram uses color coding to represent different components: red for input/output, green for layout estimation, blue for object detection, and orange for model fitting. The caption explains that the model is trained with different reward functions as described in the paper's sub-section on the proposed method.



The figure illustrates a single-agent reinforcement learning framework for training a policy to control a simulated human character in a 3D environment. The global layout is a closed-loop system with a central neural network architecture composed of two main components: a Policy network and a Value network, both represented as multi-layered gray blocks with crisscrossed internal connections, indicating deep neural networks. These networks receive inputs from a simulator and output actions and value estimates, respectively. On the left, a 'Simulator' block displays a sequence of images showing a human character walking, labeled as 'World State'. The Policy network takes the World State as input and outputs an action a_t , which is fed into the 'Single Integrator Dynamics' block. The Value network takes the World State as input and outputs a value estimate v_t , which is fed into the 'Lee Controller' block. The 'Single Integrator Dynamics' block outputs a new World State S_{t+1} , which is fed back into the Policy network. The 'Lee Controller' block outputs a 'Truncated Sparsity' signal, which is fed into the 'Single Integrator Dynamics' block. The 'Single Integrator Dynamics' block also receives a 'Reward Module' output, which is fed into the 'PPO Loss' block. The 'PPO Loss' block outputs a 'PPO Loss' signal, which is fed into the 'On-Policy Data' block. The 'On-Policy Data' block outputs a 'PPO Loss' signal, which is fed into the 'PPO Loss' block. The 'PPO Loss' block outputs a 'PPO Loss' signal, which is fed into the 'PPO Loss' block. The diagram uses color coding to represent different components: red for input/output, green for layout estimation, blue for object detection, and orange for model fitting. The caption explains that the model is trained with different reward functions as described in the paper's sub-section on the proposed method.

